Project JKeep

From Legacy Hardware to Enterprise-Grade Private Cloud

A Technical Case Study on Debian 13 (Trixie), ZFS Integrity, and Container Orchestration

Juanfe Lozano
Universidad Tecnológica de Pereira (UTP)

February 2026

Abstract

This report chronicles the transition of a consumer-grade Asus X441U laptop from a Windows 10 environment to a high-availability, headless Debian 13 NAS. The project, titled **JKeep**, implements industry-standard data integrity through the ZFS filesystem, secure orchestration via Docker, and Zero Trust networking via Tailscale. The following pages detail every technical decision, hardware constraint, and architectural optimization taken to build a professional server infrastructure from legacy hardware.

Contents

1 Introduction & Hardware Genesis **4**
1.1 The Philosophy of Project JKeep 4
1.2 Hardware Inventory: The Asus X441U 4
1.3 The "Windows 10" Dilemma 5

2 Infrastructure Strategy **6**
2.1 Operating System Selection: Why Debian? 6
2.2 Power Consumption & Economic Analysis 6
2.3 The "Pro" Stack vs. The Amateur 7
2.4 Kernel Optimization Strategy 7

3 Security & Data Integrity Theory **8**
3.1 The Zero Trust Architecture 8
3.2 Data Integrity: The "Self-Healing" Filesystem 8
3.3 The 3-2-1-1-0 Backup Rule 9

4 OS Deployment & Initial Hardening **10**
4.1 The Installation Phase 10
4.1.1 Partitioning Strategy 10
4.2 Post-Install Hardening: The Sudoers Lockout 10
4.3 Headless Optimization: GUI Neutralization 11
4.4 Energy Efficiency: Lid and Screen Persistence 11

5 Storage Engineering: The ZFS Engine **12**
5.1 Initialization and The "Flagship" Pool 12
5.2 Pool Management and the "Identity" Crisis 12
5.3 Persistence in Headless Environments 13

6 Filesystem Optimization & Data Protection **14**
6.1 Performance Tuning for Mechanical Media 14
6.2 Memory Management: ARC Tuning 14
6.3 Automated Protection: Sanoid Snapshots 14

7 Container Orchestration: The Docker Layer **16**
7.1 The Philosophy of Containerization 16
7.2 Docker-ZFS Integration 16
7.3 Remote Development with VS Code 16

8 Database Management & Service Isolation **17**
8.1 SQL vs. NoSQL: The Relational Choice 17
8.2 PostgreSQL Internal Architecture 17
8.3 The Professional Hybrid Model 17

9 The Media Cloud: Jellyfin Deployment **19**
9.1 The "Home YouTube" Experience 19
9.2 Hardware-Accelerated Transcoding 19
9.3 Security & Isolation 19

10 Remote Ecosystem & Final Hardening **20**
10.1 Samba (SMB): Finder Integration 20
10.2 Lifecycle Automation: Zsh Functions 20
10.3 Conclusion: The Professional Headless State 20

Chapter 1

Introduction & Hardware Genesis

1.1 The Philosophy of Project JKeep

The modern digital landscape is increasingly dominated by proprietary cloud services (Google Drive, AWS, Dropbox). While convenient, these services introduce risks regarding privacy, data ownership, and subscription fatigue. Project JKeep was conceived as a "Private Cloud" alternative—a server that lives in the physical control of the owner while providing the same "anywhere-access" experience.

1.2 Hardware Inventory: The Asus X441U

Repurposing old hardware is not merely a cost-saving measure but an exercise in efficient system architecture. The target machine is an **Asus X441U** laptop, which, while obsolete for modern gaming or heavy video editing, possesses ideal specs for a home NAS:

- **CPU:** Intel Core i5-6198DU @ 2.30GHz (Dual-core, 4 threads). This is a U-series (Ultra-low voltage) processor with a TDP of only 15W.
- **RAM:** 8GB DDR4 (Sufficient for basic ZFS caching and Docker containers).
- **OS Storage:** 1TB ADATA SU630 SSD (The "Operating Plane").
- **Bulk Storage:** 1TB Toshiba Mechanical HDD (The "Data Plane").
- **Network:** Integrated 10/100 Ethernet and Atheros QCA9565 Wi-Fi.

Core Concept: Laptop vs. Rack Server

Why use a laptop as a NAS? 1. **Integrated UPS:** The laptop battery acts as a built-in Uninterruptible Power Supply (UPS). If the power fails, the server can shut down gracefully. 2. **Power Efficiency:** Laptops are designed to sip power, unlike enterprise rack servers that can draw 500W+.

1.3 The "Windows 10" Dilemma

The laptop originally ran Windows 10 Home. A core decision was whether to keep Windows and run the server in a VM (Virtual Machine) or wipe it for a Bare Metal Linux install.

- **The Amateur Path:** Keeping Windows 10 and using WSL2 or a VM. This leads to "Update Hell," high RAM overhead, and telemetry-related reboots.
- **The Professional Path:** Bare Metal Debian. This eliminates all bloatware and provides the OS with direct access to the storage hardware.

Technical Insight & Warning: Windows 10 Restoration

For those worried about losing their license: modern Windows licenses are tied to the motherboard's Hardware ID. You can wipe the drive, install Debian, and later reinstall Windows 10—it will activate automatically as soon as it hits the internet.

Chapter 2

Infrastructure Strategy

2.1 Operating System Selection: Why Debian?

In the world of servers, stability is the only metric that matters. Debian was chosen over Ubuntu or CentOS for several reasons:

1. **Minimalism:** The "NetInst" allows for a headless install with zero GUI overhead.
2. **The "Gold Standard":** Most enterprise Docker images are based on Debian.
3. **Longevity:** Debian "Stable" releases are supported for years, making it a "set and forget" system.

2.2 Power Consumption & Economic Analysis

As a professional project, JKeep includes a rigorous analysis of electricity costs. Using the Intel RAPL (Running Average Power Limit) sensors and local energy rates in Colombia, we calculated the cost of 24/7 operation.

- **Idle Consumption:** 8-12 Watts.
- **Active Load (Transcoding):** 25-32 Watts.
- **Cost per kWh:** 213 COP.

Table 2.1: Projected Monthly Energy Cost (Colombia)

Mode	Avg Watts	Monthly kWh	Cost (COP)
24/7 Active	18W	12.96	$2,760
16h/Day	18W	8.64	$1,840
Extreme Idle	8W	5.76	$1,227

2.3 The "Pro" Stack vs. The Amateur

To ensure this project meets professional industry standards, we mapped our choices against "Hardcore Purist" and "Amateur" targets:

Table 2.2: Infrastructure Comparison Table

Layer	Professional (JKeep)	Amateur	Purist Perspective
OS	Debian Stable	Windows 10	"Minimal or nothing."
Filesystem	ZFS	NTFS/ext4	"ZFS with ECC RAM."
Access	Tailscale (WireGuard)	TeamViewer	"SSH Tunnel only."
Containers	Docker Compose	Manual Install	"Podman (Rootless)."

2.4 Kernel Optimization Strategy

During the installation, we opted for the `generic` kernel over the `targeted` one.

- **Reasoning:** The generic kernel includes drivers for a wider range of hardware. If the Asus motherboard fails, the SSD can be moved to another machine and boot immediately (Hardware Portability).

Technical Insight & Warning: Non-Free Firmware

Debian is strictly open-source, but hardware often is not. We enabled the `non-free-firmware` repository to ensure the Atheros Wi-Fi and Intel microcode were correctly loaded for stability and security patches.

Chapter 3

Security & Data Integrity Theory

3.1 The Zero Trust Architecture

In a professional environment, security is no longer about "walls" but about "identity." Traditional networking relies on opening ports in a firewall—a practice that leaves a server exposed to automated brute-force attacks within minutes. Project JKeep adopts a **Zero Trust** model.

Core Concept: Zero Trust Networking

Zero Trust assumes that the local network is potentially compromised. Instead of trusting any device on the LAN, every connection must be authenticated and encrypted. We implement this using **Tailscale**, a mesh VPN based on the **WireGuard** protocol.

By utilizing Tailscale, the JKeep server has zero open ports on the public internet. It establishes an outbound encrypted tunnel to a private "Tailnet," allowing access from authenticated devices only. This architectural choice is critical for home servers that lack the enterprise-grade hardware firewalls found in data centers.

3.2 Data Integrity: The "Self-Healing" Filesystem

Standard filesystems like NTFS or ext4 are "blind" when data corruption occurs. If a bit flips on a physical platter (bit-rot), these systems will serve the corrupted data without notice, leading to database crashes or corrupted media files.

Core Concept: Bit-Rot and Checksumming

ZFS (Zettabyte File System) utilizes end-to-end checksumming. Every time data is written, a mathematical hash is stored. When the data is read, ZFS re-calculates the hash; if it doesn't match, it identifies the corruption. In a mirrored setup, it would automatically repair the bit from the healthy copy.

3.3 The 3-2-1-1-0 Backup Rule

Professional data management follows a strict hierarchy of redundancy to survive even catastrophic hardware failures or ransomware attacks:

- **3** copies of data (Original + 2 Backups).
- **2** different media types (e.g., SSD for OS, HDD for backup).
- **1** copy off-site (Cloud storage like Backblaze or Google Drive).
- **1** copy offline/immutable (Protected against network-based ransomware).
- **0** errors (Verified via automated restoration tests).

Chapter 4

OS Deployment & Initial Hardening

4.1 The Installation Phase

The deployment utilized the **Debian 13 (Trixie)** Network Installer. This allowed for a hyper-minimal footprint, as the installer only pulls packages necessary for a headless environment, reducing the attack surface.

4.1.1 Partitioning Strategy

We implemented a **Manual Partitioning** scheme on the 1TB SSD to ensure the "Operating Plane" (Root) and "Data Plane" (Home) remained independent. This prevents a runaway log file from filling the disk and crashing the system.

1. **ESP (1GB):** EFI System Partition for modern UEFI booting.
2. **Root (/) (32.9GB):** The "Engine Room" where the OS binaries reside.
3. **Swap (8.5GB):** Sized to match the physical RAM for efficient memory swapping.
4. **Home (/home) (437.7GB):** The persistent data layer for user configurations.

4.2 Post-Install Hardening: The Sudoers Lockout

A common hurdle in professional Debian installs is the handling of the `sudo` group. If a root password is provided during the installation, the first user account is **not** granted administrative privileges by default.

Technical Insight & Warning: The Sudoers Lockout

Attempting to run `sudo apt update` immediately after installation results in: *"juanfe is not in the sudoers file. This incident will be reported."* **The Correct Path:** 1. Switch to root: `su -` 2. Install sudo: `apt install sudo` 3. Add user: `usermod -aG sudo juanfe` 4. Refresh: Log out and log back in.

4.3 Headless Optimization: GUI Neutralization

To ensure maximum RAM availability for the PostgreSQL database, we removed the graphical display manager entirely.

- **Purge Command:** `apt purge gdm3 gnome-shell task-desktop`
- **Text-Mode Enforced:** `systemctl set-default multi-user.target`

Technical Insight & Warning: The Blinking Cursor Deadlock

Running the purge command while inside a graphical session causes the screen to freeze. This is not a crash, but the loss of the display manager. **The Solution:** Use `Ctrl+Alt+F2` to switch to a TTY console and finish the cleanup via `apt autoremove -purge`.

4.4 Energy Efficiency: Lid and Screen Persistence

For a 24/7 NAS, we must override the laptop's default "Sleep on Lid Close" behavior.

```
# Masking sleep targets to keep the 'brain' awake
systemctl mask sleep.target suspend.target hibernate.target
```

To prevent screen burn and save energy, we implemented **Console Blanking** by adding `consoleblank=60` to the GRUB configuration. This ensures the physical screen turns off after 60 seconds of inactivity while the server continues to handle remote requests.

Chapter 5

Storage Engineering: The ZFS Engine

5.1 Initialization and The "Flagship" Pool

The core of Project JKeep's data resilience is the **ZFS (Zettabyte File System)** pool. Initializing ZFS on a legacy laptop with a mechanical HDD via USB 3.0 presents unique challenges, primarily regarding hardware "handshakes" and partition alignment.

Technical Insight & Warning: The "64M" and "Device Busy" Deadlock

During the first attempt to create the pool `tank`, the kernel threw a cryptic error: *"zfs: invalid for parameter 'zfs_arc_max'"*. Furthermore, the ZFS utility reported the 1TB drive as having "less than 64MB" of space. **The Root Cause:** The drive contained a legacy `ext4` partition table that "locked" the sectors ZFS needed for its own labels. **The Resolution:** A low-level header wipe was required to neutralize the disk.

```
dd if=/dev/zero of=/dev/sdb bs=1M count=100  # Wipe first 100MB
partprobe /dev/sdb                           # Force kernel update
zpool create -f -o ashift=12 tank [DEVICE_ID]
```

The `ashift=12` flag is critical, as it aligns ZFS blocks with the 4K sector size of modern high-capacity HDDs, preventing a massive performance degradation known as "Write Amplification."

5.2 Pool Management and the "Identity" Crisis

A significant learning moment occurred when the pool was successfully imported but appeared under the name `storage_pool` instead of the documented `tank`. ZFS stores pool metadata on the disks themselves, meaning the name is "sticky."

Technical Insight & Warning: Renaming Pools on Import

To align the hardware with the project's "Infrastructure as Code" documentation, we performed a renamed import:

```
zpool export storage_pool              # Safely detach the pool
zpool import storage_pool tank         # Re-attach with the target name
```

5.3 Persistence in Headless Environments

On a headless server, storage must be available immediately upon boot without manual intervention. We achieved this by generating a `zpool.cache` file and embedding it into the Initial RAM Disk (`initrd`).

Chapter 6

Filesystem Optimization & Data Protection

6.1 Performance Tuning for Mechanical Media

Mechanical HDDs are the bottleneck of any NAS. To mitigate the latency of the TOSHIBA 1TB drive, we applied several "Flagship" optimizations:

- **LZ4 Compression:** `zfs set compression=lz4 tank/jkeep`. By compressing data in RAM before it hits the slow USB bus, we effectively increase the perceived write speed while saving space.
- **Atime Disable:** `zfs set atime=off tank/jkeep`. This prevents the HDD from performing a "write" operation every time a file is "read," significantly reducing mechanical wear and tear.

6.2 Memory Management: ARC Tuning

ZFS defaults to using 50% of system RAM for its Adaptive Replacement Cache (ARC). On an 8GB laptop also running Docker, this can lead to "Memory Pressure" and OOM (Out Of Memory) kills.

Core Concept: ARC (Adaptive Replacement Cache)

The ARC is a "Smart Cache" that predicts which data will be needed next based on frequency and recency. We capped the ARC at 4GB to ensure the PostgreSQL database always has a guaranteed 4GB of RAM available for its own operations.

6.3 Automated Protection: Sanoid Snapshots

To survive accidental deletions or data corruption, we implemented **Sanoid**. This utility automates the creation and pruning of ZFS snapshots.

- **Policy:** Hourly snapshots kept for 36 hours, Daily for 30 days.
- **Verification:** `zfs list -t snapshot`.

Technical Insight & Warning: The Partition Table CRC Mismatch

During a troubleshooting phase, the pool became unimportable due to a "CRC Mismatch" in the primary GPT header. **The Solution:** We used `gdisk` to recover the "Main" header from the "Backup" header stored at the end of the disk. This highlighted the importance of ZFS's redundant labeling—labels are stored at both the beginning and end of the drive.

Chapter 7

Container Orchestration: The Docker Layer

7.1 The Philosophy of Containerization

In a professional DevOps environment, applications are not installed directly onto the host OS. Instead, they are "Containerized."

Core Concept: Docker as a Shipping Container

Imagine a cargo ship. Before standard containers, loading loose sacks of flour and barrels of oil was a nightmare. If the oil leaked, it ruined the flour. Docker containers act as standardized boxes. The "Slave" node (Asus) is the ship, and the applications (PostgreSQL, Jellyfin) are the containers. They share the ship's engine (Linux Kernel) but remain isolated from each other.

7.2 Docker-ZFS Integration

A key architectural decision was the mapping of Docker volumes to ZFS datasets. This ensures that the ephemeral nature of a container does not risk the persistent nature of the data.

- **Host Path:** `/tank/jkeep/docker/postgres_data`
- **Container Path:** `/var/lib/postgresql/data`

By mapping these paths, the PostgreSQL database benefits from ZFS's LZ4 compression and automated snapshots without the container even being aware of the underlying filesystem.

7.3 Remote Development with VS Code

Managing a headless server is traditionally done via SSH in a terminal. However, Project JKeep leverages the **VS Code Remote SSH** extension. **High-Level:** Your Mac acts as the "Screen," and the Asus acts as the "Brain." **Low-Level:** VS Code installs a small "Server" component on the Debian node, allowing for real-time file editing, integrated terminal access, and extension support directly on the server's hardware.

Chapter 8

Database Management & Service Isolation

8.1 SQL vs. NoSQL: The Relational Choice

For the "alCampo" and "JKeep" initiatives, we selected **PostgreSQL** (Relational) over NoSQL alternatives like MongoDB.

- **Structured Logic:** In a project involving products, users, and transactions, the relationships are "strict." SQL enforces these relationships, ensuring that a "Price" cannot exist without a "Product."
- **ACID Compliance:** Atomicity, Consistency, Isolation, and Durability. PostgreSQL guarantees that even if the power fails in Pereira, the database remains in a consistent state.

8.2 PostgreSQL Internal Architecture

PostgreSQL does not store data in human-readable files. It uses a complex binary structure optimized for speed.

- **WAL (Write Ahead Logs):** Every transaction is logged to a journal before being written to the actual table. This is the ultimate defense against corruption during a power failure.
- **Segments:** Data is broken into 1GB binary segments.

8.3 The Professional Hybrid Model

We implemented a hybrid storage strategy:

1. **Metadata (SQL):** Descriptions, prices, and user IDs are stored in the relational database.
2. **Blobs (ZFS Files):** Large files like pixel art, videos, and images are stored as raw files in `/tank/media`.

3. **The Link:** The database contains a column with the file path (`/tank/media/photo.png`), bridging the two layers.

Technical Insight & Warning: The Samba Security Gap

While we use Samba (SMB) to "drag and drop" files from a Mac Finder, the `tank/db` folder is **never** shared. **The Risk:** Accessing "live" database files via a network share can cause binary locks and corruption. Only the Docker container should have access to the raw DB files.

Chapter 9

The Media Cloud: Jellyfin Deployment

9.1 The "Home YouTube" Experience

Jellyfin was selected as the flagship media service to host family videos and marketing assets. Unlike proprietary services, Jellyfin provides a visual, poster-based interface without data mining or subscription fees.

9.2 Hardware-Accelerated Transcoding

A critical challenge with using an Asus X441U is its limited CPU power. When streaming a high-resolution video to a phone with slow mobile data, the server must "shrink" (transcode) the video.

Core Concept: Intel QuickSync (QSV)

We leveraged the Intel Integrated Graphics (iGPU) via `/dev/dri`. By passing this device into the Docker container, the hardware handles the video mathematics, leaving the CPU free for other tasks.

9.3 Security & Isolation

The Jellyfin container is mounted with the `:ro` (Read-Only) flag on the media volumes. This prevents any exploit in the media player from being used to delete or modify the original family archives on the ZFS pool.

Chapter 10

Remote Ecosystem & Final Hardening

10.1 Samba (SMB): Finder Integration

To bridge the gap between a Mac workstation and a Linux server, we implemented **Samba**. This allows the server to appear in the macOS Finder as a network drive.

- **The Unified Root:** Instead of sharing individual folders, we share the root `/tank`. This provides a single entry point for Projects, Media, and Backups, preventing "Ghost Drive" clutter on the Mac desktop.

10.2 Lifecycle Automation: Zsh Functions

To manage the server efficiently from a Mac, we created a suite of custom Zsh functions:

1. `wake-server`: Sends a "Magic Packet" (WoL) to the Asus motherboard.
2. `open-server`: Establishes a passwordless SSH tunnel via Tailscale.
3. `sleep-server`: Remotely triggers a graceful shutdown.

10.3 Conclusion: The Professional Headless State

Project JKeep successfully transformed a consumer laptop into an enterprise-grade node. By combining **ZFS integrity**, **Docker orchestration**, and **Zero Trust networking**, we have created a platform that is secure, resilient, and ready for high-level software development.

Chapter 11

Client Integration & Multi-Modal Access

11.1 The Tailscale Tunnel: The Infinite LAN

The foundation of our remote access is the **Tailscale** mesh VPN. Regardless of whether the Mac is on the same Wi-Fi as the server or on a public network across the world, the server is always accessible via its fixed Tailnet IP (e.g., `100.x.y.z`). This creates an "Infinite LAN" where services are always a single hop away without the need for complex port forwarding.

11.2 Terminal Access: SSH and Automation

Native terminal access is achieved via **SSH (Secure Shell)**. By generating an RSA key pair on the Mac and authorizing it on the server, we eliminated the need for manual password entry.

- **Primary Command:** `ssh juanfe@100.x.y.z`
- **Automation:** Using the custom `open-server` Zsh function allows for instant connectivity with a single command.

11.3 Integrated Development: VS Code Remote

For coding and configuration, we utilize the **VS Code Remote SSH** extension. This allows the Mac to act as a "Thin Client." The user interface runs locally on macOS, while the actual file system, terminal, and language compilers (Python, C++) run directly on the Asus server hardware. This prevents local Mac resources from being consumed by heavy server tasks.

11.4 File System Integration: Samba & Finder

To manage files with the ease of a local drive, we utilize **Samba (SMB)**.

- **Mounting:** In Finder, the user connects to `smb://100.x.y.z`.

- **The "SlaveServer" Volume:** This unified mount allows for "Drag and Drop" between the Mac and the ZFS pool.
- **Workflow:** A video can be edited on the Mac and dropped directly into the `/tank/media` folder, where Jellyfin immediately detects it.

11.5 Media Surveillance Access

Finally, high-level service access is achieved through dedicated client applications:

1. **Jellyfin Media Player (Mac):** Provides a high-fidelity cinematic experience without the limitations of a web browser.
2. **Termius (Mobile):** Used for emergency monitoring and server health checks from an iPhone.
3. **Web UI:** Accessing administrative dashboards via `http://100.x.y.z:8096`.